Smart convenience management system

PROBLEM

- Manual inventory management is inefficient and time-consuming
- Product searching becomes slower as product quantity increases
- Businesses require efficient data management for inventory operations
- Difficulty managing stock availability
- Slow customer order processing
- Challenges in tracking sales records

OBJECTIVE

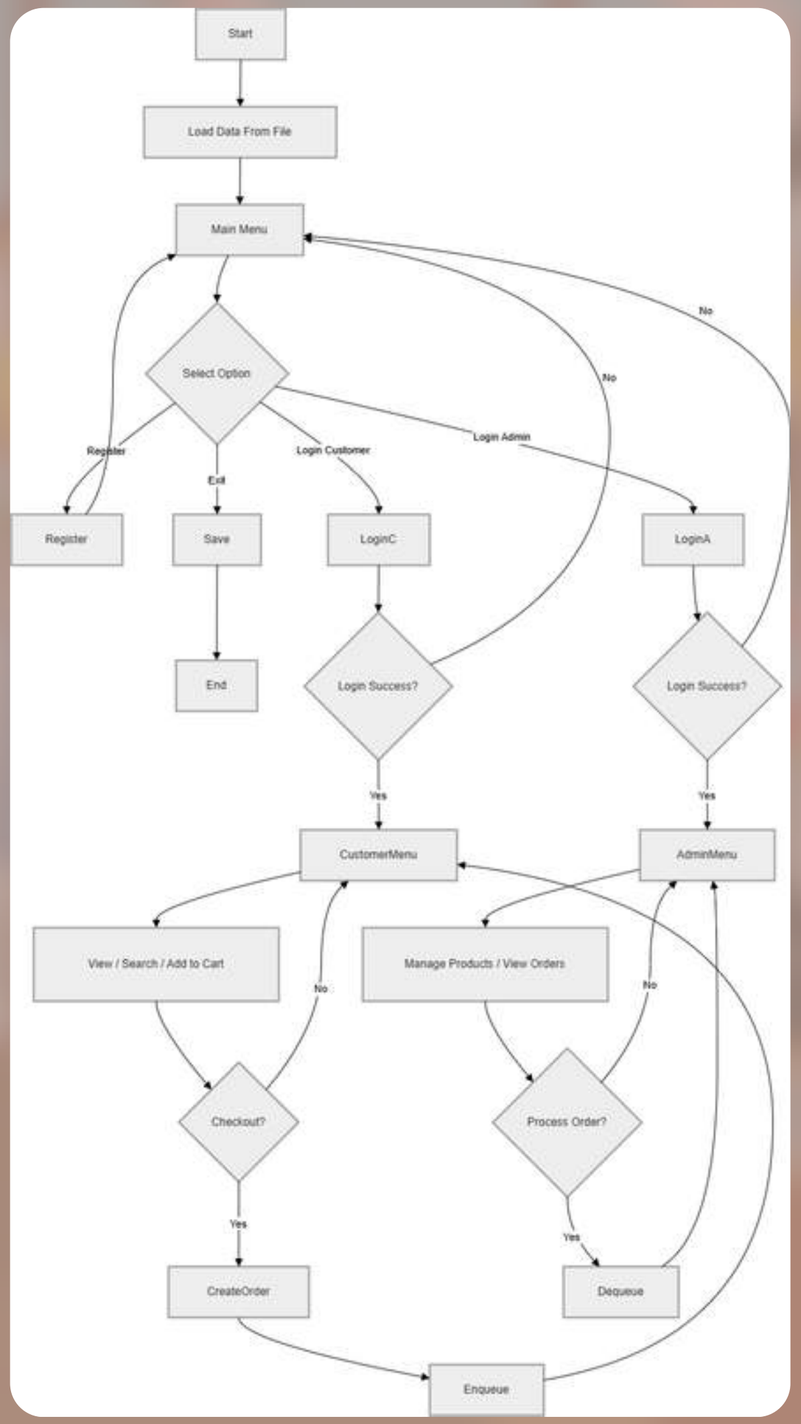
- To simplify inventory management in convenience stores
- To reduce errors in stock and order management
- To improve the speed of product searching and order processing
- To provide an organized system for managing products and customers
- To support sales tracking and product recommendations

System Features

- Add product
- Show product
- Update product
- Delete product
- Sort product
- Sale report
- Show order
- Process next order

- Show Product
- Search Product
- Add to Cart
- Show Cart
- Checkout

FLOWCHART


Start
Load Data From File
Main Menu
Select Option
Register
Exit
Login Customer
Login Admin
No
No
Register
Save
LoginC
LoginA
End
Login Success?
Login Success?
Yes
Yes
CustomerMenu
AdminMenu
View / Search / Add to Cart
Manage Products / View Orders
No
No
Checkout?
Process Order?
Yes
Yes
CreateOrder
Dequeue
Enqueue

Linked List

Used for :

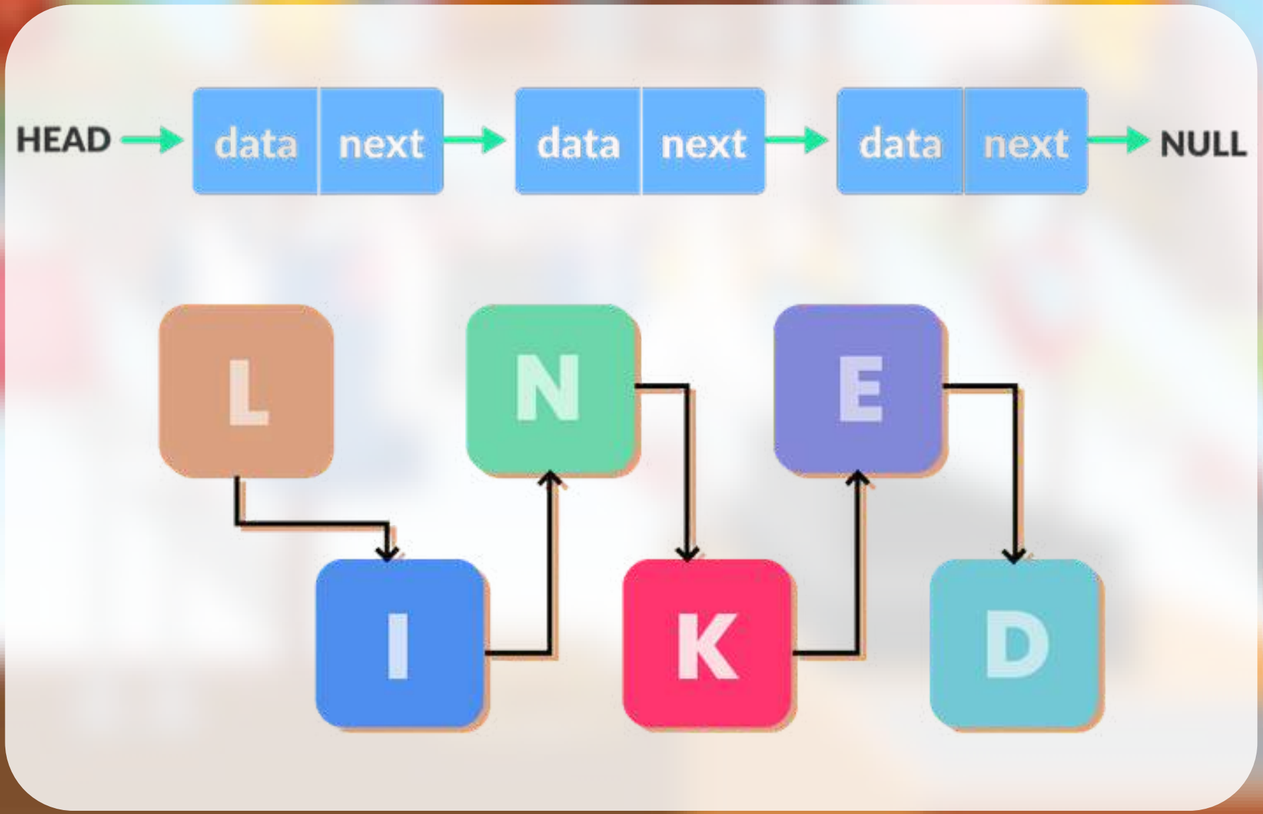
- **Fast storing for managing product data dynamically.**

Main Purpose:

- **To manage product data using dynamic memory allocation instead of fixed-size arrays.**

Why Linked List:

- **We choose a Linked List because the number of products in the system can increase or decrease at any time.**
- **Linked List allows easy insertion and deletion without shifting elements like an array.**



Queue

Used For:

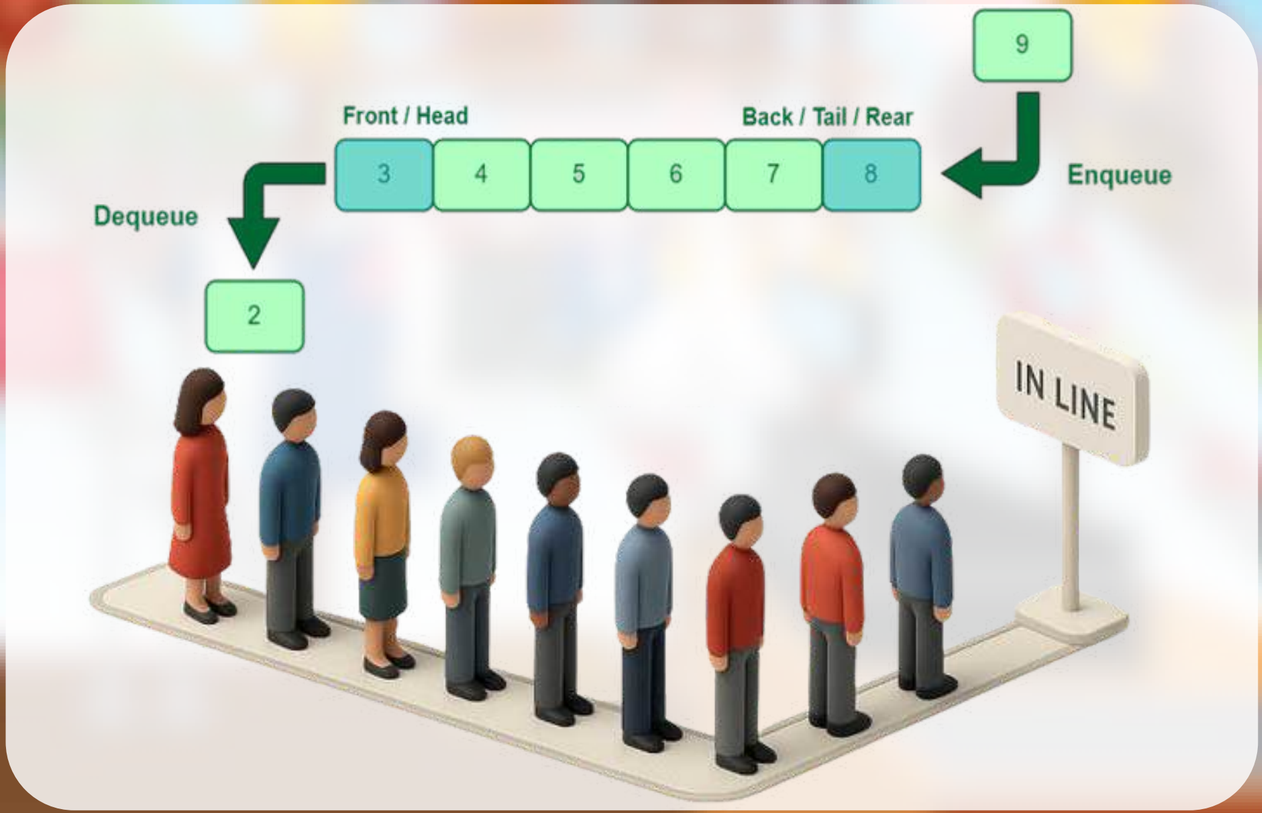
- **Managing customer orders in the system.**

Main Purpose:

- **To process orders using the First-In First-Out (FIFO) principle.**

Why Linked List:

- **New orders are added to the rear, and processed from the front.This is like simulation of convinient store that the people who come first will checkout first.**



Hash Table

Used For:

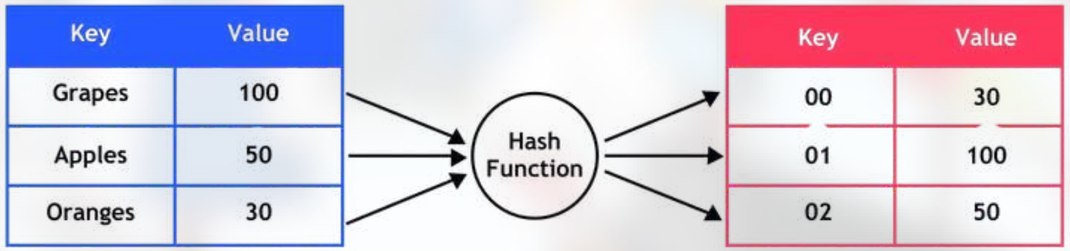
- User account storage.
- Fast username lookup.

Main Purpose:

- Faster login and registration.
- Efficient user searching.

Why Hash Table?

- Converts username into index.
- Reduces search time.
- Faster than linear searching.



Key Code

Linked List

```c
Node* newNode = createNode(p);

if(head == NULL) {
    head = newNode;
}
else {
    Node* temp = head;
    while(temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}
```

```c
Node* createNode(Product p) {
    Node* newNode = malloc(sizeof(Node));
    newNode->data = p;
    newNode->next = NULL;
    return newNode;
}
```

we implement a Linked List using a Node structure.
Each node contains product data and a pointer to the next node.

The addProduct() function creates a new node and attaches it to the end of the list.
If the list is empty, the new node becomes the head.

Queue

```c
typedef struct {
    Order orders[MAX];
    int front;
    int rear;
} Queue;
```

```c
Order* temp = front;

front = front->next;

if(front == NULL)
    rear = NULL;

free(temp);
```

Dequeue

```c
if(front == NULL) {
    front = rear = newOrder;
}
else {
    rear->next = newOrder;
    rear = newOrder;
}
```

Enqueue

we implement a Queue using the FIFO principle

The Queue structure uses front and rear pointers.
Enqueue adds a new order to the rear.
Dequeue removes an order from the front for processing.

Key Code

Hash Table

```c
#define TABLE_SIZE 100

typedef struct {
    char username[50];
    char password[50];
    int role;
} User;

typedef struct UserNode {
    User data;
    struct UserNode* next;
} UserNode;

UserNode* hashTable[TABLE_SIZE];
```

Structure Used in Code

```c
int hash(char* str) {

    int sum = 0;

    for(int i = 0; str[i] != '\0'; i++) {
        sum += str[i];
    }

    return sum % TABLE_SIZE;
}
```

Hash function

- The system converts characters in the username into ASCII values.
- All values are added together.
- The result is divided by TABLE_SIZE.
- The remainder becomes the index of the hash table.

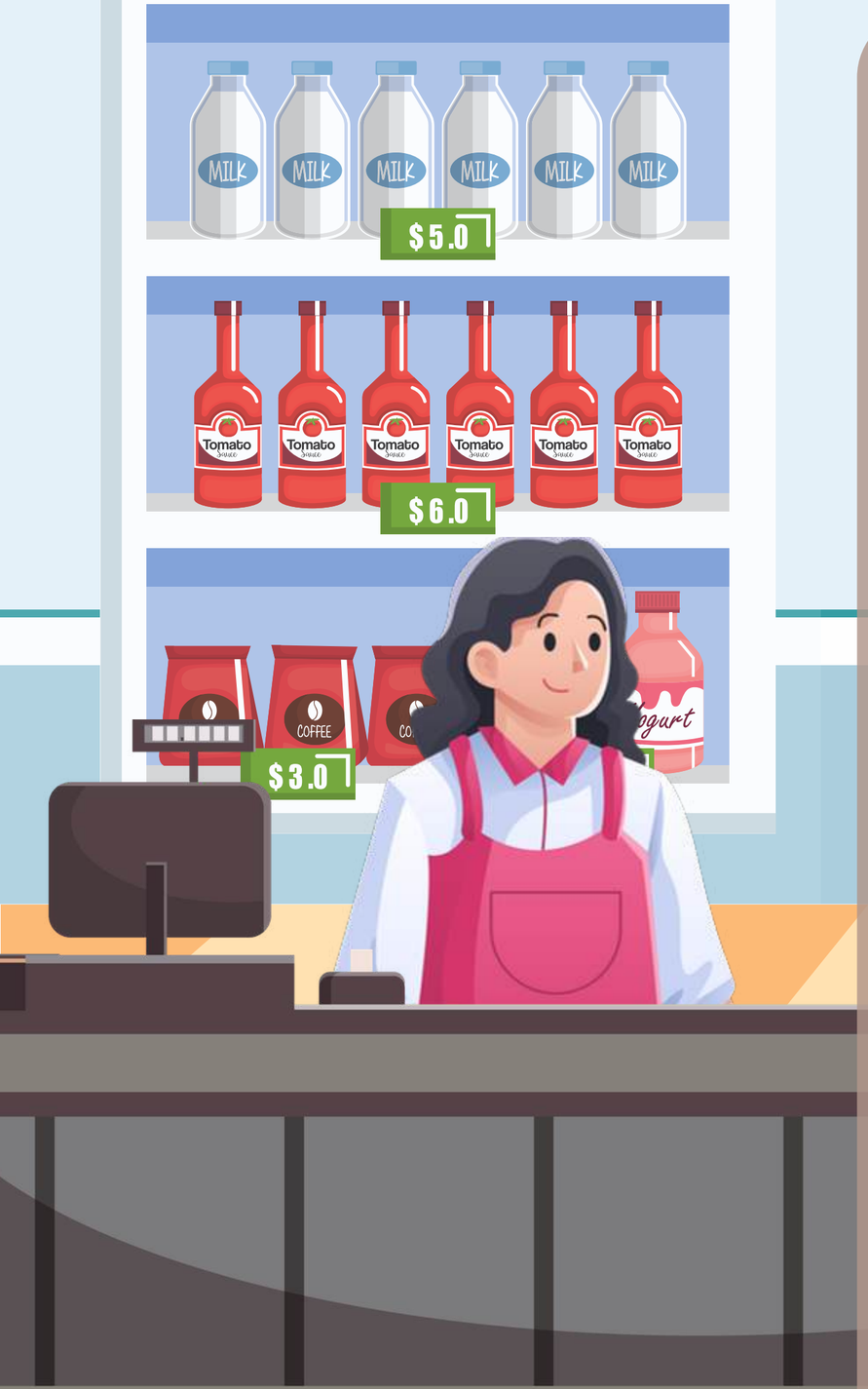
Challenges & Improvements

Challenges

- Implementing multiple data structures in one system
- Handling hash table collisions using linked list chaining
- Maintaining data consistency in text files
- Input validation and debugging
- Integrating file persistence with system operations
- Managing order processing using queue structure

Improvements

- Replace text files with database systems
- Improve recommendation algorithm
- Support multiple admin accounts
- Add advanced searching and sorting algorithms
- Improve system scalability and performance
- Add sales analytics dashboard
- Enhance security and user management



Conclusion

- Developed a Smart Convenience Management System using C language.
- Applied Linked List, Queue, and Hash Table in a real-world application.
- Implemented product management, shopping cart, and order processing features.
- Improved searching efficiency using hash tables.
- Gained experience in dynamic memory management, file handling, and debugging.
- Built a strong foundation for future development in algorithms and database systems.

Q & A

CPE112

MEMBERS
THANK YOU!

Thanpisit Ritpetchnil

68070503427

Supakorn Polphichai

68070503457

Thiti Hongkittiyanont

68070503488

Peerapat Rakcharoen

68070503444